Dr. Devi Priya K

Final_DOC

agri

Document Details

Submission ID

trn:oid:::3618:134746160

Submission Date

Apr 10, 2026, 9:33 AM GMT+5:30

Download Date

Apr 10, 2026, 9:40 AM GMT+5:30

File Name

Final_DOC.docx

File Size

686.0 KB

54 Pages

10,025 Words

59,825 Characters

38% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

- **243** Not Cited or Quoted 38%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **4** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

25% Internet sources
21% Publications
33% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- **243** Not Cited or Quoted 38%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **4** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

25% Internet sources
21% Publications
33% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Student papers	Graphic Era University on 2026-01-15	13%
2	Internet	lbrce.ac.in	3%
3	Publication	Narisetty Srinivasarao, Tiyyagura Chandra Reddy, Kokkiligadda Pushpa Latha, Kill...	1%
4	Internet	jseepublisher.com	<1%
5	Student papers	UCL on 2025-08-27	<1%
6	Internet	www.coursehero.com	<1%
7	Student papers	aou on 2024-12-21	<1%
8	Student papers	Curtin University of Technology on 2021-09-29	<1%
9	Publication	Pranab Dey. "Computational Cytology", Springer Science and Business Media LLC,...	<1%
10	Internet	pdfcoffee.com	<1%

11 Student papers

University of Warwick on 2026-01-19 <1%

12 Student papers

Taylor's Education Group on 2025-07-25 <1%

13 Student papers

University of Bucharest on 2026-02-06 <1%

14 Internet

albrittonanalytics.com <1%

15 Student papers

Mohamad Bin Zayed University of Artificial Intelligence on 2023-04-29 <1%

16 Publication

Pradeep Singh, Balasubramanian Raman. "Deep Learning Through the Prism of T... <1%

17 Internet

www.rapidinnovation.io <1%

18 Internet

www.gpcet.ac.in <1%

19 Student papers

Manipal University Jaipur Online on 2025-01-26 <1%

20 Student papers

University of St Andrews on 2025-08-11 <1%

21 Internet

dione.lib.unipi.gr <1%

22 Internet

ebin.pub <1%

23 Student papers

University of Wales Institute, Cardiff on 2023-05-09 <1%

24 Internet

johal.in <1%

25 Internet

theses.liacs.nl <1%

26 Internet

web.realinfo.tv <1%

27 Student papers

Berlin School of Business and Innovation on 2025-10-12 <1%

28 Student papers

CSU, San Jose State University on 2022-10-18 <1%

29 Student papers

University of Wales Institute, Cardiff on 2026-02-08 <1%

30 Student papers

Imperial College of Science, Technology and Medicine on 2023-03-17 <1%

31 Student papers

Liverpool John Moores University on 2020-04-26 <1%

32 Student papers

University of Bucharest on 2026-02-06 <1%

33 Student papers

University of North Texas on 2025-10-13 <1%

34 Student papers

Hong Kong University of Science and Technology on 2025-05-01 <1%

35 Student papers

UNIBA on 2025-11-10 <1%

36 Student papers

University of Hertfordshire on 2025-01-06 <1%

37 Student papers

University of Wales Institute, Cardiff on 2023-03-05 <1%

38 Internet

discuss.pytorch.org <1%

39 Internet

www.analyticsvidhya.com <1%

40 Student papers

Information Science and Technology on 2025-08-22 <1%

41 Student papers

University of Hertfordshire on 2025-09-13 <1%

42 Internet

mdpi-res.com <1%

43 Student papers

Gujarat Technological University on 2026-03-24 <1%

44 Internet

bmcmedinformdecismak.biomedcentral.com <1%

45 Student papers

Columbia University on 2026-03-26 <1%

46 Student papers

University of Adelaide on 2024-06-26 <1%

47 Student papers

Higher Education Commission Pakistan on 2023-07-08 <1%

48 Student papers

Islington College,Nepal on 2026-03-30 <1%

49 Internet

eprints.utm.my <1%

50 Internet

www.geeksforgeeks.org <1%

51 Publication

Suresh Ukarande, Shailesh R. Nikam, Makarand G. Kulkarni, Nandana A. Prabhu e... <1%

52 Student papers

University of East London on 2025-09-08 <1%

53 Student papers

University of Hertfordshire on 2025-04-27 <1%

54 Internet

dfzljdn9uc3pi.cloudfront.net <1%

55 Internet

pmc.ncbi.nlm.nih.gov <1%

56 Internet

www.jihmsp.org <1%

57 Student papers

Information Science and Technology on 2025-08-09 <1%

58 Student papers

The Robert Gordon University on 2026-03-22 <1%

59 Student papers

University of Hertfordshire on 2024-06-19 <1%

60 Internet

earchive.tpu.ru <1%

61 Internet

www.tericsoft.com <1%

62 Student papers

University of Keele on 2025-01-05 <1%

63 Student papers

University of Melbourne on 2025-11-07 <1%

64 Student papers

University of New Haven on 2025-11-17 <1%

65 Student papers

University of Wales, Lampeter on 2023-05-02 <1%

66 Student papers

Yeditepe University on 2025-12-29 <1%

67 Internet

www.semanticscholar.org <1%

68 Student papers

Indian Institute of Information Technology, Allahabad on 2025-12-13 <1%

69 Publication

Paolo Ferro, Harinadh Vemanaboina, Chander Prakash. "Computational Techniqu... <1%

70 Student papers

University of Carthage on 2023-06-15 <1%

71 Internet

nou.edu.ng <1%

72 Internet

www.akgec.ac.in <1%

73 Student papers

University of Bucharest on 2026-02-06 <1%

74 Internet

dspace.bracu.ac.bd <1%

75 Internet

ijritcc.org <1%

76 Student papers

kitsw on 2025-04-14 <1%

77 Internet

www.ijraset.com <1%

78 Internet

3dgeo-heidelberg.github.io <1%

79 Publication

Alex Khang. "AI-Powered Cybersecurity for Banking and Finance - How to Enhanc... <1%

80 Student papers

Cyberjaya University College of Medical Sciences on 2025-01-09 <1%

81 Student papers

Middle East Technical University on 2026-02-11 <1%

82 Student papers

Universitas Muslim Indonesia on 2024-03-28 <1%

83 Student papers

University of Adelaide on 2025-06-08 <1%

84 Student papers

University of Glasgow on 2025-10-31 <1%

85 Student papers

University of Hertfordshire on 2025-06-19 <1%

86 Student papers

University of Hertfordshire on 2026-01-05 <1%

87 Internet

iksigma.blogspot.com <1%

88 Internet

2020machinelearning.medium.com <1%

89 Publication

Abeer Saad Alsheddi, Mohamed El Bachir Menai. "Writing style change detection: ... <1%

90 Student papers

Colorado School of Mines on 2024-10-28 <1%

91 Student papers

Computer College on 2016-05-09 <1%

92 Student papers

Illinois Institute of Technology on 2025-12-06 <1%

93 Publication

S.P. Jani, M. Adam Khan. "Applications of AI in Smart Technologies and Manufactu... <1%

94 Publication

Srinivas Sethi, Bibhudatta Sahoo, Deepak Tosh, Suvendra Kumar Jayasingh, Soura... <1%

95 Student papers

Tower Hamlets College on 2023-05-26 <1%

96 Student papers

University of Hertfordshire on 2025-04-27 <1%

97 Internet

download.bibis.ir <1%

98 Internet

openi.pcl.ac.cn <1%

99 Internet

www.mdpi.com <1%

100 Internet

www.theseus.fi <1%

101 Student papers

Anna University on 2025-08-05 <1%

102 Student papers

De Montfort University on 2025-11-04 <1%

103 Student papers

Intercollege on 2022-05-05 <1%

104 Student papers

Somaiya Vidyavihar on 2024-12-21 <1%

105 Student papers

University of Central Florida on 2025-03-02 <1%

106 Student papers

University of Houston System on 2025-09-23 <1%

107 Student papers

University of Sydney on 2025-09-14 <1%

108 Internet

github.com <1%

109 Internet

kindatechnical.com <1%

110 Internet

pacehackweek.github.io <1%

111 Internet

www.chw.edu.au <1%

112 Publication

Arvind Dagur, Sohit Agarwal, Dhirendra Kumar Shukla, Shabir Ali, Sandhya Sharm... <1%

113 Student papers

Heriot-Watt University on 2025-04-07 <1%

114 Student papers

Islington College,Nepal on 2026-01-06 <1%

115 Student papers

Islington College,Nepal on 2026-01-07 <1%

116 Publication

Meher Afroj, M. Rubaiyat Hossain Mondal, Md Riad Hassan, Sworna Akter. "MobD... <1%

117 Publication

Natasa Kleanthous, Abir Hussain. "Machine Learning in Farm Animal Behavior usi... <1%

118 Publication

Shen Wen, Shuang Yang, Ling Xu, Yan Yang, Yangzhi Qi, Ping Hu, Qianxue Chen, ... <1%

119 Student papers

Singapore Institute of Technology on 2023-11-06 <1%

120 Publication

Thangaprakash Sengodan, Sanjay Misra, M Murugappan. "Advances in Electrical ... <1%

121 Student papers

University of Bucharest on 2026-02-06 <1%

122 Student papers

University of Hertfordshire on 2025-04-28 <1%

123 Student papers

University of North Texas on 2023-12-14 <1%

124 Student papers

University of Warwick on 2023-09-25 <1%

125 Student papers

VNR Vignana Jyothi Institute of Engineering and Technology on 2025-12-27 <1%

126 Internet

arxiv.org <1%

127 Internet

bright-journal.org <1%

128 Internet

krs.chmnu.edu.ua <1%

129 Internet

open-innovation-projects.org <1%

130 Internet

ubir.buffalo.edu <1%

131 Internet

wjarr.com <1%

132 Internet

www.fabriziomusacchio.com <1%

133 Internet

www.jneonatalsurg.com <1%

134 Student papers

Coventry University on 2025-04-11 <1%

135 Publication

Parag Verma, Er. Devarasetty Purna Sankar, Anuj Bhardwaj, Vaibhav Chaudhari, ... <1%

136 Student papers

University of New Haven on 2025-04-30 <1%

137 Student papers

University of Westminster on 2026-02-16 <1%

138 Publication

Zhehat Rebar, Adnan Mohsin Abdulazeez. "Deep and Machine Learning Algorith... <1%

139 Publication

Δημητρακόπουλος, Ηλίας. "Αξιολόγηση μεθόδων Ταξινόμησης και Ερμηνευσιμότ... <1%

140 Publication

"Data Science, AI and Applications", Springer Science and Business Media LLC, 2025 <1%

141 Student papers

Anna University on 2026-02-21 <1%

142 Student papers

Berlin School of Business and Innovation on 2026-02-20 <1%

143 Publication

Dasgupta, Subhram. "Enhancing Deepfake Identification With CNN and Multi-hea... <1%

144 Student papers

Jawaharlal Nehru Technological University on 2009-10-16 <1%

145 Publication

Mehdi Ghayoumi. "Generative Adversarial Networks in Practice", CRC Press, 2023 <1%

146 Student papers

Mepco Schlenk Engineering college on 2025-04-11 <1%

147 Student papers

UCL on 2025-09-08 <1%

148 Student papers

University of Sunderland on 2024-08-11 <1%

A REAL TIME, RESOLUTION-AGNOSTIC AI FRAMEWORK FOR ENHANCED GLIOMA DETECTION AND SEGMENTATION

A Project Report

submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE & ENGINEERING

Submitted by

Chintada Yaswanth	22761A0511
Kancharana Hari Prasad	22761A0524
Kalakonda Giridhara Venkata Krishna Sai	23765A0502

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

LAKIREDDY BALI REDDY COLLEGE OF ENGINEERING

(AUTONOMOUS)

Accredited by NAAC with 'A' Grade & NBA (Under Tier - I)

An ISO 21001:2018, 14001:2015, 50001:2018 Certified Institution

Approved by AICTE, New Delhi and Affiliated to JNTUK, Kakinada

L.B. REDDY NAGAR, MYLAVARAM, NTR Dist., ANDHRA PRADESH – 521230

2022-2026

LAKIREDDY BALI REDDY COLLEGE OF ENGINEERING

(An Autonomous Institution since 2010)
Accredited by NAAC with 'A' Grade & NBA (Under Tier - I),
An ISO 21001:2018, 14001:2015, 50001:2018 Certified Institution
Approved by AICTE, New Delhi and Affiliated to JNTUK, Kakinada
L.B. REDDY NAGAR, MYLAVARAM, NTR DIST., A.P.-521 230.
hodcse@lbrce.ac.in, cseoffice@lbrce.ac.in, Phone: 08659-222 933, Fax: 08659-222931

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

CERTIFICATE

This is to certify that the project entitled "**A REAL TIME, RESOLUTION-AGNOSTIC AI FRAMEWORK FOR ENHANCED GLIOMA DETECTION AND SEGMENTATION**" is being submitted by

Chintada Yaswanth	**22761A0511**
Kancharana Hari Prasad	**22761A0524**
Kalakonda Giridhara Venkata Krishna Sai	**23765A0502**

in partial fulfillment of the requirements for the award of degree of **B. Tech** in **Computer Science & Engineering** from **Jawaharlal Nehru Technological University, Kakinada** is a record of bonafide work carried out by them at **Lakireddy Bali Reddy College of Engineering.**

The results embodied in this Project report have not been submitted to any other University or Institute for the award of any degree or diploma

Mr. N. Srinivasa Rao	**Dr. Nagarjuna Reddy**
(Sr. Asst. Professor)	**(Head of the Department)**

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We express our sincere gratitude to our respected **Principal**, **Dr. K. Appa Rao**, for creating an academic environment that fosters innovation, research, and knowledge-sharing.

We extend our heartfelt thanks to **Dr. S. Nagarjuna Reddy**, **Professor & Head of the Department**, CSE for providing the necessary resources and a conducive environment for research and learning.

We extend our deepest appreciation to my project guide, **Mr. N. Srinivasa Rao**, **Sr. Assistant Professor,** for their invaluable guidance, constant support, and insightful feedback throughout this project. Their encouragement and expertise have been instrumental in shaping my work.

We are also grateful to **Ms. B. Swathi, Sr. Assistant Professor**, Project Coordinator, for their continuous support and timely advice, which helped us stay on track and complete the project effectively.

We would also like to acknowledge the **teaching and non-teaching staff** of CSE department, whose guidance and assistance have been of great help throughout our academic journey.

A special thanks to our **family and friends** for their unwavering support, patience, and encouragement during the course of this project. Their belief in us has been a great source of motivation.

Lastly, we are grateful to everyone who has directly or indirectly contributed to the successful completion of this project.

Chintada Yaswanth - 22761A0511
Kancharana Hari Prasad - 22761A0524
Kalakonda Giridhara Venkata Krishna Sai - 23765A0502

DECLARATION

We hereby declare that the project report entitled "**A REAL TIME, RESOLUTION-AGNOSTIC AI FRAMEWORK FOR ENHANCED GLIOMA DETECTION AND SEGMENTATION**" is submitted to JNTUK is partial fulfillment of the requirement for the award of the degree of bachelor of technology is an original work carried out by us. The matter embodied in this Project report is a genuine work by the students and has not been submitted earlier to this university or any other university for award of any degree or diploma or prize.

Signatures of the students

Chintada Yaswanth - 22761A0511
Kancharana Hari Prasad - 22761A0524
Kalakonda Giridhara Venkata Krishna Sai - 23765A0502

ABSTRACT

Detecting glioma - among the fiercest kinds of brain growths - demands precision, often leaning on MRI data examined closely by experts. Yet human review takes hours, opens room for inconsistency, particularly when image quality dips into blur or graininess. Errors creep in more easily under such conditions. This pushes demand higher for tools that work fast, stay consistent, ignore resolution limits, without needing constant oversight. Here, a streamlined deep-learning model steps forward: built to catch tumor signs instantly across varied scan qualities. Its core adapts VGG-16, reshaped with side-running filters running at once, pulling out details at different scales smoothly. Fewer repeated calculations occur because pathways share insights smartly within the structure. From the same core network, two distinct pathways emerge - one tailored for detecting tumors using a modified YOLO structure, the other refining boundaries through an upgraded HDSA mechanism. What sets this approach apart lies in its Resolution Agnostic Loss, a flexible scoring method adjusting itself depending on how clear or blurry the input scan appears. Rather than treating all images the same, it weighs tasks differently when resolution shifts, keeping outputs stable even with poor-quality data. Performance remains robust across varied MRI sources, especially noticeable on lower-grade scans where many systems struggle. Testing took place on BraTS, a standard collection used to gauge progress in brain imaging research. Findings show strong scores in both spotting tumors and outlining their shapes accurately, all without slowing down processing time. This equilibrium of quick processing alongside precise outcomes positions the framework well for use in clinics, since fast judgments often matter there. The tool demonstrates clear promise for healthcare workers - lightening their evaluation burden while boosting uniformity and accelerating dependable brain tumor analysis.

LIST OF CONTENTS

1. INTRODUCTION 01

1.1. Overview of the Project 01-03

1.2. Feasibility Study 04

1.3. Scope 05

2. LITERATURE SURVEY 06

91 2.1. Existing System & Drawbacks 09-10

2.2. Proposed System & Advantages 11

3. SYSTEM ANALYSIS 12

3.1. Overview of System Analysis 12-13

3.2. Software used in the project 14-15

3.3. System Requirements 16

36 **4. SYSTEM DESIGN** 17

4.1. Overview of System Design 17-27

5. CODING & IMPLEMENTATION 28-38

6. SYSTEM TESTING 39

7. RESULTS 40-42

8. CONCLUSION 43

9. REFERENCES 44-45

10. APPENDICES 46

49

List of Figures

Figure No.	Figure title	Pg.No
1	Software Development Life Cycle	12
2	Architecture	17
3	Augmented images	21
4	Kernel	22
5	Stride	23
6	Padding	24
7	Activation Function	25
8	Flattening	26
9	VGG-16 Training Loss VS Validation Loss	41
10	VGG-16 Training Accuracy VS Validation Accuracy	41
11	Confusion Matrix for VGG-16	42
12	ROC Graph for VGG-16	42

31

List of Tables

Table No.	Title of the table	Pg.No
1	Accuracy and loss for the model: CNN	40
2	Accuracy and loss for the model: VGG-16.	40

List of Abbreviations

1. MRI – Magnetic Resonance Imaging
2. CNN - Convolutional Neural Network
3. VGG – Visual Geometry Group
4. DL – Deep Learning
5. YOLO – You Look Only Once
6. HDSA – Hierarchical Dual-Stream Attention
7. RAL – Resolution-Agnostic Loss
8. ReLU – Rectified Linear Unit
9. ROC – Receiver Operating Characteristic
10. IoU – Intersection over Union
11. AUC – Area Under Curve
12. BraTS – Brain Tumor Segmentation
13. CT – Computed Tomography
14. ML – Machine Learning
15. T1 – T1 - Weighted MRI
16. T2 – T2 - Weighted MRI
17. T1CE – T1 – Weighted Contrast Enhanced
18. FLAIR – Fluid Attenuated Inversion Recovery

1. INTRODUCTION

1.1 Overview of The Project

Glioma is a type of brain tumor that starts in glial cells, which help support and protect neurons in the brain. It is one of the most serious and aggressive forms of brain tumors because it grows quickly and spreads into nearby brain tissue. Gliomas are usually classified by their grade. Low-grade tumors, like astrocytomas, grow slowly but can become more severe over time. High-grade tumors, such as glioblastoma (GBM), grow rapidly and are much more dangerous, often leading to a poor prognosis. One of the biggest challenges in treating glioma is that it spreads into healthy brain tissue, making complete surgical removal very difficult. Because of this, early detection and accurate diagnosis are extremely important for improving patient outcomes[1].

Early on, individuals might notice persistent head pain, sudden fits, trouble seeing clearly, problems concentrating, or minor coordination hiccups. When the mass expands, effects tend to intensify - memory lapses appear, behavior shifts occur, standing steady gets harder, moving around becomes awkward, sometimes leading to weakness on one side. Because such warning signals overlap with many nerve-related disorders, relying just on visible clues often leads to uncertainty. That's when detailed scans step in, offering clearer insight needed to confirm what is happening inside.

Though MRI stands as the top choice for spotting brain tumors, its strength lies in showing soft tissue clearly. Because different types - like T1, T2, FLAIR, and those using contrast - offer varied views, they reveal details about shape, volume, and location. Yet despite high accuracy, reading these pictures demands skill, often consuming hours. Image inconsistencies between scanners, along with noise levels, further complicate interpretation. Each machine's quirks may alter clarity, adding challenge to consistent results[2].

Looking at MRI images by hand takes considerable time, yet mistakes happen more often than expected. Depending on who reviews them, outcomes shift - shaped heavily by individual skill and perception, while tiny tumors slip through unnoticed now and then.

Confirmation[3] Sometimes tissue samples are used, though cutting into the body carries risk and cannot apply universally. Because of such issues, machines stepping in makes sense: they offer faster judgments without losing precision or uniformity across cases.

Deep learning lately made strong progress analyzing medical images. CNN-based systems, including structures like VGG, ResNet, or U-Net, spot and outline brain tumors in MRIs quite successfully[4]. By combining multiple model styles, accuracy gets a boost through richer feature capture. Still, several of these approaches turn out too intricate, depend heavily on clear visuals, while struggling when inputs are blurry or distorted. Real clinics often face imperfect imaging setups - this reality makes deployment harder than expected.

Facing such challenges, this effort centers on designing a deep learning setup capable of operating instantly while adjusting smoothly to varying image clarity[5]. Efficiency paired with flexibility defines the intended model, enabling stable results regardless of where input originates. By doing so, the divide shrinks - between what labs develop and what clinics actually need. Progress here aligns innovation with everyday medical reality.

Earlier work examined various deep learning methods applied to glioma analysis. While U-Net architectures dominate segmentation tasks, CNNs and ResNet variants show strong performance in classifying tumors. Fast processing needs have pushed YOLO-inspired designs into broader use due to their speed advantages. Although combining different models can boost precision, such integration typically demands more computing power. This added load may limit deployment where response time is critical[6].

One way to look at this work is through its ability to spot brain tumors instantly, followed by outlining their shape, judging how serious they are, then giving clear reasons behind each decision. Built on a modified version of VGG-16, the method adds new routes inside the network to better grasp key image details. Because it includes a special loss mechanism - adjusting smoothly regardless of image resolution - it performs consistently even when scans vary in clarity. Despite running complex tasks quickly, it avoids heavy computation loads. What stands out most is how straightforward clinicians may find using it during actual patient care. Rather than prioritizing complexity, speed and trustworthiness guide every part of its structure[7].

1.1 Feasibility Study

This project has been evaluated from different angles to check whether it is practical and worth implementing. Overall, it is technically possible, cost-effective, and beneficial for society. The main goal is to improve how accurately and quickly glioma tumors can be detected and segmented from MRI scans, helping both doctors and patients. A feasibility study simply checks if a system can be built and used effectively in real life. Here, we look at three main aspects:

1.1.1 Economical Feasibility

The economic feasibility of the project depends on the cost required for its development and implementation. The system is mainly based on open-source tools such as Python, TensorFlow, and PyTorch, which reduces software costs significantly. However, training deep learning models may require GPU-enabled systems or cloud-based resources, which can involve some initial investment.

1.1.2 Technical Feasibility

The technical feasibility of the project depends on the availability of required technologies and resources. The proposed system uses well-established deep learning architectures such as VGG-16, YOLO-based detection, and segmentation techniques, which can be implemented using existing frameworks.

1.1.3 Social Feasibility

From a social point of view, the system is highly useful and acceptable. It is designed to support doctors, not replace them, by helping them detect tumors more quickly and accurately. This can lead to better treatment decisions and improved patient outcomes. It can also be especially helpful in areas where access to experienced radiologists is limited. Overall, the system has a positive impact on healthcare and is likely to be well accepted.

1.2 Scope

This project focuses on building a real-time deep learning system that can detect and segment glioma tumors from MRI scans, even when the image quality varies. The main idea is to support radiologists by giving quick, accurate, and reliable insights into brain tumor images.

The system uses different types of MRI scans like T1, T2, FLAIR, and contrast-enhanced images to get a complete view of the tumor. This helps in understanding its size, shape, and location more clearly. It is built on a modified VGG-16 model that extracts important features, along with separate components for detecting tumors and outlining their boundaries.

One of the key strengths of this system is the Resolution-Agnostic Loss (RAL), which allows it to perform consistently even when MRI images come in different resolutions or qualities. To improve performance, several preprocessing steps are used, such as normalization, noise removal, skull stripping, resizing, and data augmentation. The model is trained and tested using the BraTS dataset.

Additionally, the system emphasizes efficiency and scalability, making it suitable for real-time applications and adaptable to different clinical environments. Its ability to handle variations in image quality makes it more practical for real-world use, especially in healthcare settings with limited resources. Faster performance comes through when the setup adjusts smoothly to heavier workloads, fitting right into live diagnostics while shifting between medical centers. What stands out is how well it manages uneven scan clarity, bringing steady results where tools and funding might be tight.

Still, the effort focuses on spotting and dividing areas, stopping short of complete medical judgment or deciding on care steps. Instead of taking over from doctors, it helps them make choices. Down the line, putting it into clinics and linking systems could come next.

2. LITERATURESURVEY

Brain tumor detection based on deep learning approaches and magnetic resonance imaging:

A. B. Abdusalomov, M. Mukhiddinov, and T. K. Whangbo introduced a deep learning approach for detecting brain tumors using the YOLOv7 model. Their method focuses on real- time detection and improves performance by adding attention mechanisms and feature pyramid networks. These additions help the model focus on the most important areas in MRI scans and capture features at different scales, which is useful since tumors can vary in size. Sometimes spotting what kind of tumor sits inside the brain falls to this tool, catching gliomas or maybe a meningioma, even those near the pituitary[8]. Speed shows up as one clear edge, especially when waiting too long could shift outcomes. Faster than older CNN styles often are, yet still holds on to solid precision without slipping much. It handles pictures quickly, skipping heavy preparation work. Because of that, it works well when speed matters, like during live tasks. With smarter design choices inside, features come out clearer. Patterns in health scans become easier to spot, even tricky ones. What stands out? Less fuss, more clarity. Yet here comes the catch. Tiny tumors sometimes slip through, particularly if hiding in blurry zones of scans. When pictures get grainy or lose contrast, the system often fumbles. Real MRIs differ wildly from one hospital to another - that inconsistency shakes trust in results. Speed does not fix everything, after all. Sharpening how it handles messy, uneven inputs remains key ahead.

Brain tumor detection and classification using fine-tuned CNN with ResNet50 and U-Net model :

A. A. Asiri worked alongside M. A. Alzahrani, S. A. Alshahrani, and S. A. Alqahtani to design a deep learning setup merging ResNet50 with U-Net, aiming at classifying and segmenting brain tumors. Instead of treating these jobs separately, they built one flow where ResNet50 pulls out rich image details using transfer learning from MRI scans - those insights then move into U-Net for precise outlining of tumor edges. Because everything runs through a unified structure, results come faster while staying aligned across stages. What makes ResNet shine here? Its shortcut paths stop gradients from fading during training, opening doors for

grasping intricate visual traits without getting stuck[9].

Trained using open data like TCGA-LGG and TCIA - collections filled with labeled MRI images - the model proved effective when tested. High marks in Dice similarity and IoU appeared after trials, a sign it handles tumor outlining well, along with sorting types correctly. Because it builds on prior learning, the system needs less time to train, yet works better once up and running.

Still, the system isn't perfect. Running it demands sharp MRI scans, something that pushes up expenses because strong computers with big memory are needed. Because of this, using it instantly in clinics gets tricky - especially where tools and funds run low. Its layered design piles on difficulty, slowing down both learning and decision speed, making wide rollout harder when used outside labs.

MRI-based brain tumor detection using convolutional deep learning methods and selected machine learning techniques :

S. Saeedi, S. Rezayi, H. Keshavarz, and S. R. N. Kalhori carried out a detailed study on brain tumor detection using MRI scans by comparing different deep learning and traditional machine learning methods. Instead of focusing on just one model, they explored a mix of approaches, including a custom 2D CNN, a convolutional autoencoder for feature extraction, and classifiers like Support Vector Machines (SVM). This kind of comparison helps in understanding which methods work better for tumor detection[10].

Among all the models they tested, the custom 2D CNN performed the best in terms of accuracy and ROC scores. It was able to learn important features from MRI images more effectively than traditional machine learning methods. The use of autoencoders also helped improve feature extraction by capturing useful patterns in the data, which further boosted the classification results. Still, the research zeroes in on sorting cases rather than pinpointing tumors or outlining their shape. What holds back real-world use is that clinicians rely heavily on knowing precisely where a growth sits and how far it spreads before deciding treatment. Without showing why it reaches certain conclusions, the system feels like a black box to those who must

act on its output. Shifts in scan clarity tend to throw off its accuracy, making outcomes less consistent. Even though classification works well here, what stands out most is the gap toward building tools that do more things at once - plus make their choices easier to follow.

42 **Brain tumor detection and classification using deep learning model and sine-cosine fitness grey wolf optimization:**

Hanaa ZainEldin alongside Samah A. Gamel tackled brain tumor identification through deep learning, aiming at sharper results without slowing things down. Radiologists usually spend long hours reviewing scans, their workload piling up due to increasing patient numbers and imaging data. Instead of following old patterns, the team built an automatic method - leaning on neural networks - to handle detection faster yet just as precisely. El-Sayed M. El-Kenawy, working with Amal H. Alharbi, pushed for systems that don't sacrifice correctness when moving quickly. Because human judgment varies, depending only on specialists can create delays. Doaa Sami Khafaga joined forces with Abdelhameed Ibrahim to refine how machines learn from images, making fewer mistakes over time. Then Fatma M. Talaat stepped in, tightening the model's ability to tell tumor types apart correctly[11].

133

A new approach named BCM-CNN uses a deep learning setup built from Inception-ResNetV2, already trained on vast image data. Instead of standard tuning methods, it applies ADSCFGWO - a smart mix of sine-cosine patterns and grey wolf behavior - to adjust key settings. That method reshapes how the network learns by refining its layout and training details at once. Learning strength grows because two strategies join: one mimics natural cycles, another copies hunting tactics in packs. Together, they push accuracy higher through smarter search steps across options.

One way the software works is by sorting brain scans into either healthy or tumor groups. Because it uses already trained networks, spotting key patterns gets easier while saving time during setup. Near-perfect performance appears in tests, hitting a score just below full precision when checked against BraTS 2021 data.

Yet details like tumor shape or exact position fall outside its main goal of sorting cases. Because of that, doctors might find it less helpful when precise outlines matter most. Even so, one clear takeaway stands out - fine-tuning methods can powerfully lift accuracy in scans.

MRI-based brain tumor image detection using CNN-based deep learning method:

Chattopadhyay alongside Maitra introduced a method blending deep learning with classic machine learning for spotting brain tumors. While built mainly around Convolutional Neural Networks, the setup pulls out useful traits straight from 2D MRI scans without human help. Since these networks excel at noticing layers of structure - like lines, surface details, plus complicated growth shapes - they beat hand-crafted methods most times. After gathering those clues, another stage kicks in: a Support Vector Machine takes over to label them accurately. Because it draws sharp dividing lines, the SVM strengthens choices made after deep extraction[12].

Accuracy was strong, suggesting combining CNNs with traditional classifiers boosts diagnosis outcomes. Instead of 3D methods, using 2D MRI slices cuts down processing demands, simplifying setup overall. Still, even with those gains, weaknesses remain present. Quality of MRI input matters greatly - one key flaw lies here. Real clinics rarely deliver perfect images; distortions from movement, graininess, or uneven sharpness commonly appear, dragging results downward. Processing begins only after multiple stages like normalizing images, removing noise, followed by isolating regions of interest.

Focused primarily on sorting rather than pinpointing tumor locations, the model falls short in tasks demanding precise boundary detection. Because clinical settings often require clear outlines for therapy design, this gap reduces practical utility. Though performance in categorization stands out - showcasing promise for combined modeling strategies - the need for highly curated inputs slows deployment across diverse environments. Heavy preparation steps before analysis hinder swift implementation where speed matters most.

An improved deep learning-based hybrid model with ensemble techniques for brain tumor detection from MRI images:

Starting with earlier efforts, M. N. Islam, M. M. Rahman, and M. A. Hossain proposed a deeper neural network structure blending several CNNs alongside LSTM units to detect brain tumors. Though built on familiar components, the system targets two types of data at once -

spatial layout and ordered sequence details - from MR images. Where traditional CNN layers pick out forms like abnormal growth outlines, brightness shifts, or surface structures, the added LSTMs track how these signs evolve across adjacent scan sections. Different CNN setups bring varied perspectives, so the system captures broader details about tumors. Because each network sees data differently, merging their results builds a more reliable overall view. This mix behaves like a team, where weaknesses in one model get balanced by others. With time-series patterns mattering in scans, adding LSTM helps link information between image layers[13].

Even so, the model brings notable demands on computation. Because it uses several CNNs alongside LSTM layers, its structure grows deep and bulky - slowing down training while needing powerful hardware like GPUs. As layer count rises, predictions take longer, which limits usefulness in urgent medical settings. Tuning many hyperparameters carefully adds another hurdle, since balancing them within such an intricate setup proves tricky.

CNN-TumorNet: Leveraging explainability in deep learning for precise brain tumor diagnosis on MRI images:

N. Rasool, M. A. Khan, A. Rehman, alongside S. Kadry, introducing a framework called CNN-TumorNet - this one built with Local Interpretable Model-agnostic Explanations (LIME) to clarify how decisions form. Since opacity remains common in deep learning models used medically, uncertainty tends to grow; such systems are frequently labeled "black boxes" due to their hidden logic. To counteract this, LIME was embedded so users receive image-based insights showing exactly which parts of an MRI influenced the outcome[14]. Because clinicians now see what drives each result, confidence in the system rises - it becomes easier to assess whether conclusions make sense. Understanding follows more naturally when evidence appears visually grounded. High accuracy in classification made the tool useful for identifying tumor versus non-tumor instances. Because clinicians trust decisions they can understand, adding explanations improved its real-world potential. Still, it only sorts images - no boundary mapping was built into the design. Drawing exact outlines around tumors matters greatly in medicine; such detail guides therapy choices, surgical steps, and follow-up scans. When models skip this step, information about dimensions, form, or position stays hidden. That lack limits how deeply it fits into day-to-day patient care routines.

2.1 Existing System & Drawbacks

The existing systems for brain tumor detection using MRI images are generally classified into two categories, they are:

- **Normal:** MRI scans without any tumor are classified as normal.
- **Tumor:** MRI scans containing abnormal tumor regions are classified as tumor.

2.1.1 Medical approach :

- Most times, spotting a brain tumor begins with an MRI - this method shows clear images of brain structure. Besides that, doctors might turn to CT or PET scans when deeper insight is needed.
- Starting off different, MRI types like T1 show tumors clearly. Meanwhile, T2 highlights areas of swelling in brain tissue. On a separate note, FLAIR images make it easier to see fluid changes. Another point - contrast scans reveal spots where tissue has died. Together, these methods map out abnormal sections inside the brain.
- Most times doctors rely on a biopsy to figure out what kind of tumor and how advanced, yet it involves cutting into tissue - comes with danger.
- Imaging tools such as DTI open new paths for seeing how tumors behave. Instead of basic scans, these approaches reveal inner details through unique signals. Spectroscopy adds another layer by showing chemical activity inside tissues. With time, patterns emerge that standard views miss entirely. Each method brings a separate kind of clarity when used together[15].

2.1.2 Technical approach :

- Existing systems use machine learning and deep learning models such as CNN, VGG, ResNet, and U-Net for tumor detection and segmentation. Preprocessing techniques like normalization, filtering, and histogram equalization are applied to improve image quality. Models like YOLO are used for fast tumor detection, while U-Net is commonly used for segmentation tasks. Traditional methods such as SVM, KNN, and autoencoders are also used for classification and feature extraction[14].

2.1.3 Drawbacks

- A number of systems lack fine-tuning for medical scans, so performance dips when images get tricky. Still, even basic setups can spot clear anomalies without extra adjustments. What matters most is how well the data matches real patient variety. Sometimes simpler designs work just as fast but miss subtle signs hidden in layers. Training on narrow datasets leads to shaky results once applied elsewhere.
- Fuzzy or grainy pictures weaken how well the system works, since sharp detail from clean MRIs shapes its success. Scans without clear edges slow everything down. When images lack depth, results lose strength too. Clarity in each slice of data sets the base for better outcomes[16].
- When boosting image contrast, methods such as histogram equalization can also amplify background graininess at the same time.
- Heavy computing demands usually come with deep learning models, so powerful machines become necessary. Hardware needs tend to climb when these systems run complex tasks.
- Autoencoders or techniques such as KNN often struggle to make good use of labeled examples, sometimes relying too much on how their settings are chosen. Sometimes these approaches waste available labels because they're sensitive to tuning choices made by users.
- Not every system handles both jobs at once - some skip combining tasks entirely. Instead of splitting efforts, they stick to just one role. A few try sorting data while others map parts separately. Most never mix these functions into a single flow. Rarely does a setup manage dual aims without extra tools.
- Most current systems struggle with blurry pictures, which makes them less useful outside labs. Though they work well in ideal setups, their performance drops when image quality is poor[17]. Since many everyday situations involve grainy visuals, this weakness becomes a big hurdle. Without clear inputs, these tools can miss key details entirely.

2.2 Proposed System

- Our proposed system is a real-time, resolution-agnostic deep learning framework for detecting and segmenting glioma tumors from MRI images. It identifies and outlines gliomas without depending on fixed image dimensions. Performance stays consistent even when scan clarity differs across inputs. The design focuses on precision despite differences in how images are captured.

- The system uses a modified VGG-16 backbone for feature extraction, enhanced with parallel convolutional modules to capture important features from MRI scans. The extracted features are passed to multi-task heads, where one head performs tumor detection and classification, and the other performs segmentation to identify tumor boundaries at the pixel level.

- A key feature of the system is the Resolution-Agnostic Loss (RAL), which helps maintain performance across different image resolutions. Preprocessing steps such as normalization, noise removal, and data augmentation are applied to improve accuracy and robustness. Overall, the system provides a fast and efficient solution suitable for real-time clinical use. A new approach processes MRI scans in real time, adapting to any resolution for tumor analysis. Built on deep learning.

Dataset:

A collection known as BraTS forms the core of this work - roughly 2,130 brain MRI images tagged with tumor details. Instead of relying on single imaging types, various modes appear: T1, T2, FLAIR, along with contrast-enhanced T1. These different views offer richer context for spotting abnormal growths inside the brain. Pixel-level labels come attached to each scan, enabling precise learning setups for identifying boundaries. To refine inputs before processing, steps including scaling, resizing, and artificial variation take place. Image clarity differs across cases, introducing real-world inconsistencies useful during training. Because of these diverse conditions, systems tested here face realistic challenges. This mix supports fair testing grounds when measuring how well new methods perform. Reliability in evaluation emerges naturally from such varied yet structured input.

3. SYSTEM ANALYSIS

3.1 Overview of System Analysis

Fig1.Software Development Life Cycle

The system analysis for this project involves understanding the problem statement, gathering requirements, designing the system, implementing the system, testing the system, and deploying the system.

77

- Planning
- Analysis
- Design
- Implementation
- Testing
- deployment
- Maintenance

3.1.2 Requisites Accumulating and Analysis

In this phase, the project requirements are analyzed, and a plan is created for the software development process. This includes defining the project scope, determining the timeline and budget, and identifying the resources required. In this phase, the requirements are further analyzed, and the software system's functional specifications are defined. The analysis phase is critical as it sets the foundation for the software development process.

3.1.3 System Design

In this phase, the software system's architecture is designed, and the software components are identified. The design phase produces a detailed technical specification that outlines the software's functionality, performance, and user interface.

3.1.4 Implementation

In this phase, the software development team writes the code for the software system. The implementation phase includes activities such as coding, testing, and debugging.

3.1.5 Testing

In this phase, the software system is tested to ensure that it meets the functional requirements and is free of defects. Testing is a critical phase of the SDLC as it helps identify and correct any issues with the software system.

3.1.6 Deployment

In this phase, the software system is deployed to the production environment. This includes activities such as installation, configuration, and data migration.

3.1.7 Maintenance

In this phase, the software system is maintained to ensure that it continues to meet the user's needs. Maintenance activities include bug fixes, performance tuning, and feature enhancements.

3.2 Software Used in The Project

The software used in this system could include:

Libraries, Modules & Packages NumPy:

NumPy (short for Numerical Python) is a Python library designed for fast mathematical operations on large amounts of data. It's mainly used to work with arrays (especially multi-dimensional ones) and provides tools for linear algebra, statistics, and more.Unlike Python lists, NumPy arrays (ndarray) store data in contiguous memory blocks, making them much faster for numerical tasks. This speed advantage is crucial in data science, machine learning, and scientific computing.

Pandas:

Pandas is an open-source Python library designed for data manipulation and analysis, offering fast, flexible, and expressive data structures like Series (1D) and DataFrame (2D) for handling structured data efficiently. Built on top of NumPy, it integrates seamlessly with libraries like Matplotlib, SciPy, and Scikit-learn, making it a core tool in data science workflows. It is ideal for working with tabular data (like spreadsheets, SQL tables), time series, and heterogeneous datasets. Pandas supports data cleaning, transformation, aggregation, and visualization in a concise and readable way.

Tensor Flow or PyTorch:

TensorFlow is an open-source machine learning framework developed by Google Brain, widely used for building, training, and deploying ML and deep learning models across platforms like desktop, mobile, web, and cloud. Its scalability, comprehensive ecosystem, and multi- language support make it suitable for both research and production environments.

Keras:

Keras is a high-level deep learning API designed to make building and training neural networks fast, simple, and flexible. Originally released in 2015 by François Chollet, it is now the official high-level API of TensorFlow, though it can also run on backends like JAX and PyTorch. Its design philosophy focuses on user-friendliness, modularity, and rapid prototyping, making it popular among researchers and developers.

Scikit-learn:

Scikit-learn (sklearn) is a widely used open-source Python library for machine learning, built on NumPy, SciPy, and Matplotlib. It provides a consistent API for tasks like classification, regression, clustering, dimensionality reduction, preprocessing, and model evaluation. Its design allows both beginners and experts to build, train, and evaluate models efficiently without deep mathematical implementation from scratch.

Matplot lib or Sea born:

Matplotlib is a powerful, open-source Python library for creating static, animated, and interactive visualizations. Originally developed by John D. Hunter, it is widely used in data science, machine learning, and scientific computing for producing high-quality plots. It supports a variety of chart types, from simple line graphs to complex 3D visualizations, and integrates seamlessly with NumPy, Pandas, and Jupyter Notebooks.

Torchvision:

TorchVision is a core library in the PyTorch ecosystem designed specifically for computer vision tasks. It provides popular datasets, pre-trained model architectures, and common image/video transformations, making it easier to build and deploy vision-based deep learning applications.

It supports image and video I/O, data augmentation, and feature extraction, and integrates seamlessly with PyTorch tensors. TorchVision also offers utilities for object detection, segmentation, classification, keypoint detection, and optical flow.m and SGD, which are used to update model weights during training for better performance.

Open CV:

OpenCV (Open Source Computer Vision Library) is a powerful open-source toolkit designed for computer vision and machine learning tasks. It provides a comprehensive set of tools to process and analyze visual data such as images and videos. Originally developed by Intel, it is now maintained by the OpenCV Foundation and a global community of contributors.

OpenCV operates by interpreting images and videos as numerical matrices. Each image is represented as a grid of pixel values, where grayscale images are stored as 2D arrays and color images as 3D arrays. These matrices are manipulated using algorithms to perform tasks like edge detection, object recognition, and image transformations.

3.3 System Requirements

3.3.2 Software Requirements

Software: Python-version: 3.8 or above, Any IDLE Shell

Libraries: NumPy, Pandas, OpenCV, PyTorch, Torchvision, Scikit-learn, Matplotlib, TQDM

Operating system: Windows, Linux

3.3.3 Hardware Requirements

Processor: Intel coreI5

Ram: 8GB

Hard Disk: 500GB or More

GPU: NVIDIA GPU (Optional but recommend for faster training process)

turnitin Page 40 of 67 - Integrity Submission Submission ID trn:oid:::3618:134746160

4. SYSTEM DESIGN

4.1 Overview of System Design

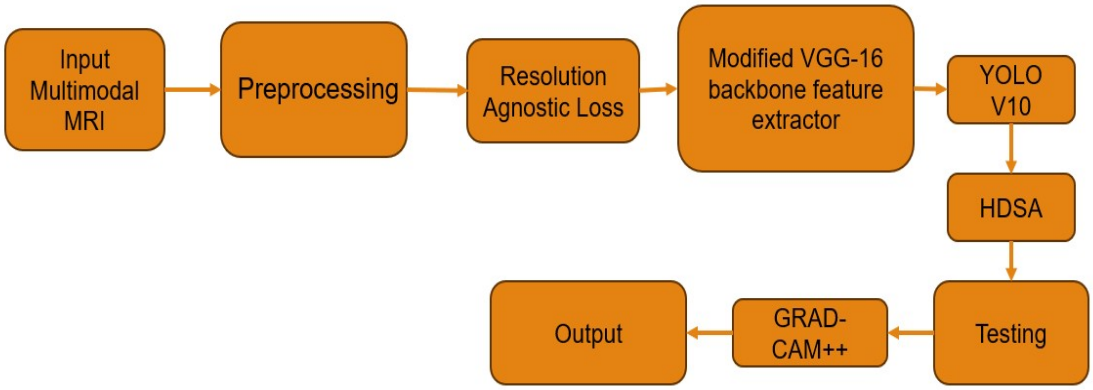


Fig 2. Architecture

The proposed system is designed as a real-time, resolution-agnostic deep learning framework for glioma detection and segmentation using multimodal MRI images. The architecture follows a sequential yet multi-branch pipeline that integrates preprocessing, feature extraction, detection, segmentation, and explainability.

The process starts with multimodal MRI data - like T1, T2, FLAIR, and T1CE - is fed into the system. Each of these types reveals distinct details regarding brain structures, tumors, or irregularities. Because they offer varied insights, combining them leads to a fuller picture than any single type alone could provide.

After the initial collection, data moves into preparation - a key phase shaping its reliability and uniformity. Starting here, methods like adjusting brightness levels come into play, followed by eliminating random interference. Another task takes out non-essential parts, focusing only on brain regions. Images shift to matching dimensions through scaling, helping balance structure across samples. Alongside, extra versions of inputs appear via controlled variations. Such steps align visual information more closely, fitting it better for learning systems. They also help models respond more reliably when faced with new examples. Once preprocessed, images move into the

turnitin Page 40 of 67 - Integrity Submission Submission ID trn:oid:::3618:134746160

Modified VGG-16 Backbone Feature Extractor. Meaningful patterns emerge from MRI data through this stage's processing. Instead of standard layers, parallel convolutions allow detection across multiple scales. Because tumor dimensions often differ, such flexibility improves recognition accuracy.

From here, the captured characteristics move into two separate pathways at once

141 YOLOv10 Detection Head This branch is responsible for fast and accurate tumor detection and classification. It identifies whether a tumor is present and localizes it within the MRI image. The use of YOLO enables real-time performance.

Tumor outlines emerge clearly through the HDSA Segmentation Head, assigning labels to individual pixels. Where precision matters most, this component maps affected areas down to fine details. Its output reveals dimensions and spatial extent - information vital for clinical assessment. Shape definition becomes possible because each region gets distinct identification. Medical evaluation leans on such accuracy when decisions depend on visual evidence.

One output adjusts through the Resolution Agnostic Loss (RAL), a method shifting smoothly with changes in image clarity or size. Because it responds to resolution differences, performance stays steady regardless of scan type. Detection and segmentation stay aligned - not separate but linked - through this balance. Reliability emerges not from rigidity, but flexibility built into the process.

Once complete, testing begins - unseen MRI scans are run through the model. Its outputs move next into Grad-CAM++, a tool revealing key image areas behind each choice. These highlighted sections make the results clearer, helping users understand how conclusions were reached.

Output emerges after processing - tumor classifications appear alongside outlined tumor zones and visual explanations. Radiologists use these elements to reach conclusions more quickly, accuracy improves as a result.

4.2 Methodology

4.2.2 Noise Reduction and Enhancement

Initially, the MRI scans get processed through noise removal steps to clear out distortions possibly hiding key data. Through the use of certain filters - like Gaussian or median types - unwanted graininess fades, yet critical features stay intact. Following that, adjustments in contrast help highlight faint variations in tissue structure, making areas likely affected by tumors more visible. Ultimately, these improvements support clearer interpretation of internal anatomy.

Gray = 0.299 × 'RED' + 0.587 × 'GREEN' + 0.114 × 'BLUE'

Here green, red, blue are the individual color channel values of a pixel and gray is the resulting grayscale intensity value for that pixel. The formula for the Gaussian blur kernel is :

$$G(a, b) = \frac{1}{2\pi\sigma^2} \times e^{-\frac{a^2+b^2}{2\sigma^2}}$$

- Here, G(a,b) is the value at coordinates (a,b) in the Gaussian kernel.
- σ represents the standard deviation of the Gaussian distribution, determining the amount of blur. Higher σ values result in more blur.
- a and b represent the distances from the center of the kernel.

4.2.2 Skull Stripping

Skull stripping is performed to remove non-brain tissues such as skull and background, allowing the model to focus only on the brain region.

$$I_{brain} = I \times M$$

Where:

• I is the original MRI image

- M is a binary mask
- I_{brain} is the extracted brain region

This step improves accuracy by eliminating irrelevant information.

4.2.3 Normalisation and standardization

Consistency across the dataset begins with adjustments to pixel intensity scales, ensuring scan-to-scan alignment. Because MRI images often differ in brightness levels, rescaling their values helps reduce unwanted variation. One method reshapes data using average and spread metrics - this is known as z-score transformation. Such shifts in value distribution allow later modeling steps to perform more reliably. Uniformity in these numeric patterns supports clearer comparisons during evaluation phases.

$$f(y) = a\,y(-y_i^3) + b\,y(-y_i^2) + c\,y - (y_i + d)$$

Here, $f(y)$ represents the interpolated value at a point y between y_i and y_{i+1}. The coefficients a, b, c, and d are determined based on the known values at y_i and y_{i+1}, along with their derivatives or slope information.

4.2.4 Data Augmentation for Model Generalization

Although few originals exist, useful variety emerges when adjustments are made with care. Through rotation, resizing, or mirroring preprocessed visuals, new appearances form naturally. Key traits stay intact even as position and scale shift across instances. Exposure to such shifts prepares learning systems for real-world conditions they might later face. Instead of fixating on precise pixel layouts, attention moves toward recurring structural clues. Better generalization emerges when models learn from varied examples. Because of augmented inputs, reliance on exact training patterns decreases. Minor shifts in data matter less once exposure increases. When new situations appear, responses stay steadier. Adaptability grows alongside consistency through repeated diversification.

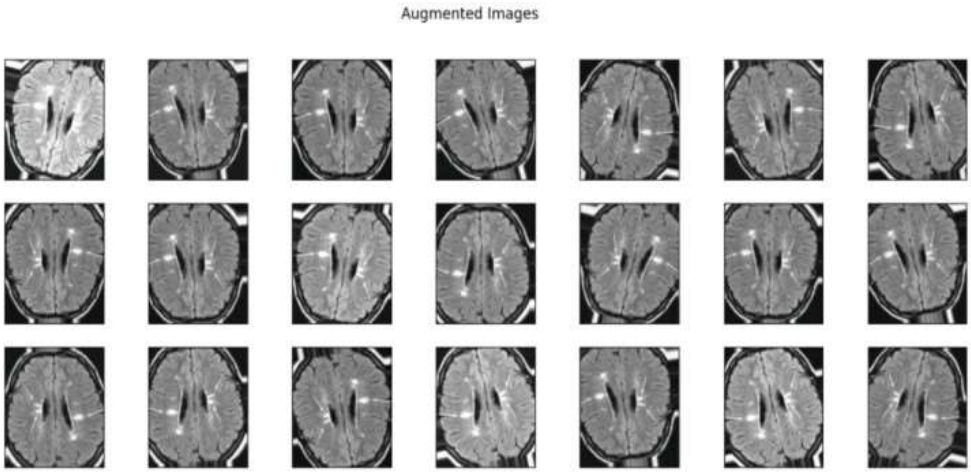


Fig 3. Augmented Images.

4.2.5 VGG-16 Components

VGG-16 model consists of different types of components:

- Convolutional Layers (Kernel / Feature Extractor)
- Stride
- Padding
- Activation Function (ReLU)
- Pooling
- Flattening
- Fully Connected Layers
- Softmax Layers

Convolutional Layers (Kernel / Feature Extractor)

The convolutional layer extracts features such as edges, textures, and patterns from the input image using filters (kernels). The kernel slides over the image and performs element-wise multiplication and summation to generate feature maps.

$$F(i,j) = \sum_m \sum_n (i+m, j+n) \cdot K(m,n)$$

Example:

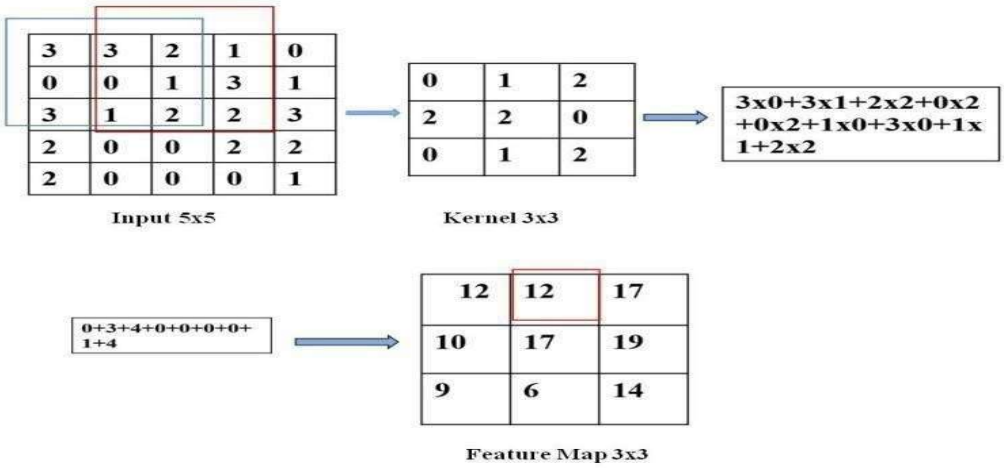


Fig 4. Kernel

Stride

The amount of filter movement is called stride. It moved across the input image from left to right, top to bottom, with one-pixel value change on the horizontal position and one-pixel change in vertical position. The default value of stride in 1-Dis1and in 2-Dis(1,1)for height and width movement.

Example

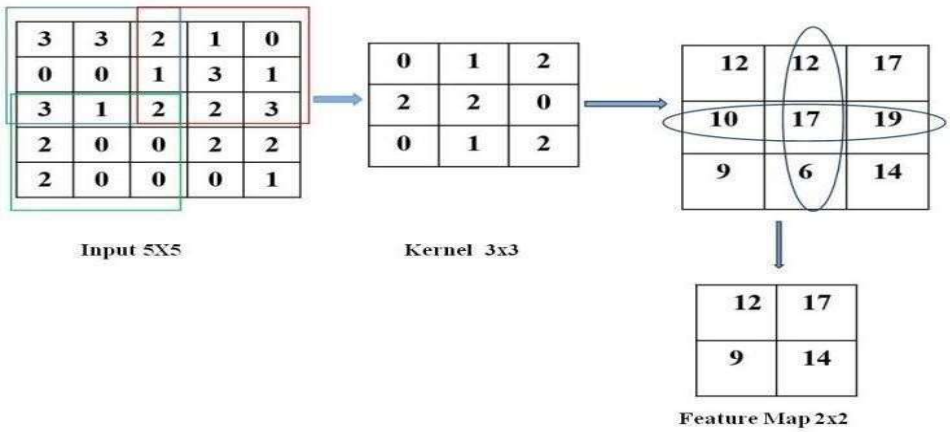


Fig 5. stride

turnitin Page 46 of 67 - Integrity Submission Submission ID trn:oid:::3618:134746160

Padding

Padding is mainly used for solving the Border Problem. It is used when the edge pixels play's an important role in classification task. If an input image is padded and then it passed into the CNN model to give the more accurate analysis of images.

Formula

$$O = \left\lfloor \frac{i-k+2p}{s} \right\rfloor + 1$$

O=output matrix size
i=input matrix size
k = kernel matrix size
s = stride value

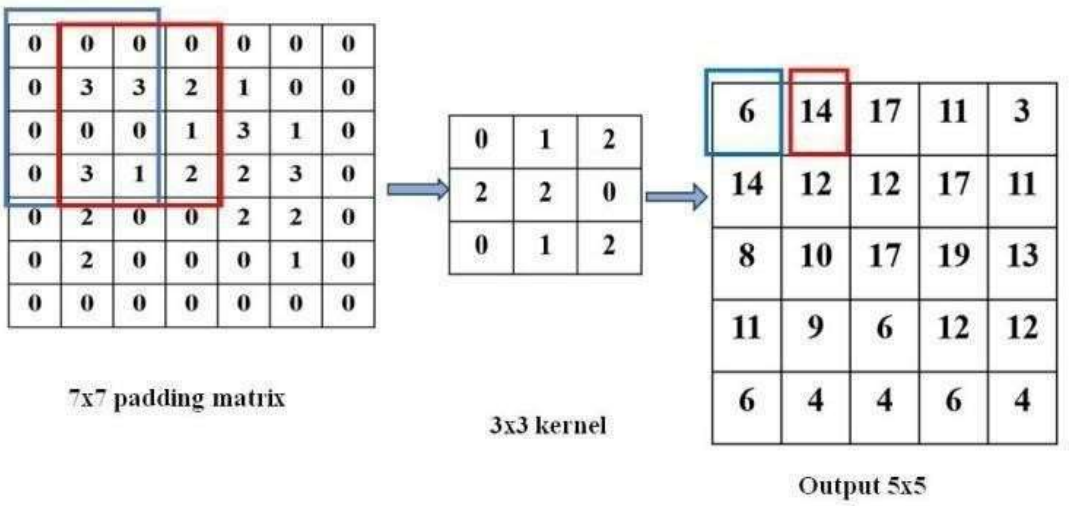


Fig 6.padding

Activation Function(ReLU):

The ReLU activation function is a piecewise linear function that outputs the input directly if it is positive; otherwise, it outputs zero. This function is mathematically described as:

$$f(x) = max\ (0, x)$$

turnitin Page 46 of 67 - Integrity Submission Submission ID trn:oid:::3618:134746160

The graph of the ReLU activation function appears as a "bump" with a negative derivative to the left of x < 0, which is a key characteristic that distinguishes it from other activation functions. The ReLU function is widely used in deep learning models due to its simplicity and efficiency, allowing neural networks to learn complex patterns while avoiding the vanishing gradient problem.

Example:

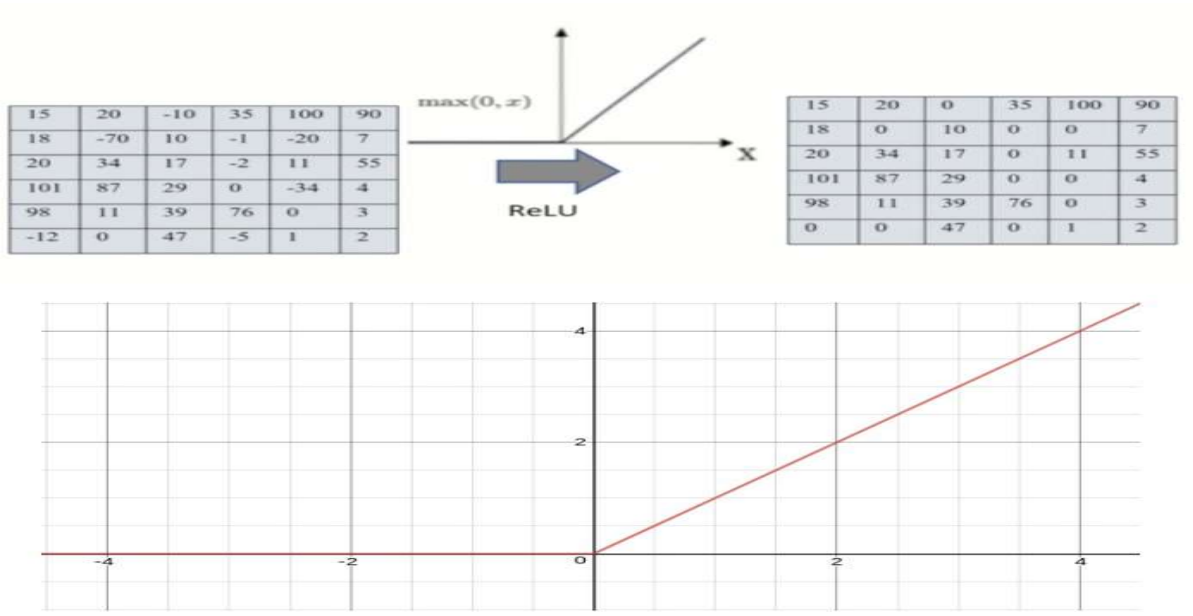


Fig 7.Activation function

Pooling

Pooling is one of the important components that makes CNN very effective. Pooling is used to reduce the size of input image size in order to reduce the computation power. It has two types of operations.

a. Max pooling

In max pooling it takes the max value from the sub matrix selected and places that max value in the output matrix.

b. Average pooling

In average pooling it calculates the average value of the sub matrix selected from the input matrix and places in the output matrix.

Flattening

A pooled feature map obtained before the flattening layer is passed into the flattening layer in order to make it into a single column matrix which is then given as an input to the neural network for processing.

Example:

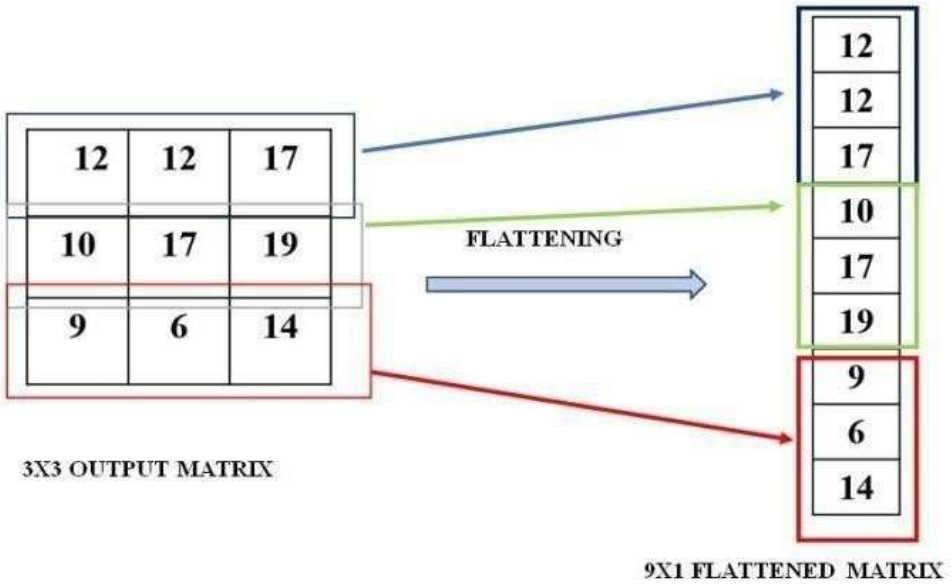


Fig 8. Flattening

Fully Connected Layer

A fully connected (FC) layer consists of neurons, weights, biases, and activation functions. Each neuron receives input from all neurons in the previous layer, multiplies each input by a corresponding weight, adds a bias term, and passes the result through an activation function such as ReLU, Sigmoid, or Tanh to introduce non-linearity. This dense connectivity allows the layer to integrate information from all features, making it essential for tasks requiring complex decision-making.

5. Coding & Implementation

Code:

```
 import cv2
import torch
import numpy as np
import pandas as pd
import torch.nn as nn
import torch.nn.functional as F
import torchvision.models as models

from torch.utils.data import Dataset, DataLoader

device = "cuda" if torch.cuda.is_available() else "cpu"

LR = 1e-4
BATCH_SIZE = 4
EPOCHS = 20
LAMBDA_DENSITY = 0.1

from google.colab import drive
drive.mount('/content/drive')

ZIP_SOURCE = "/content/drive/MyDrive/Glioma/BraTS_New.zip"
ZIP_DEST = "/content/BraTS_New.zip"

!cp "$ZIP_SOURCE" "$ZIP_DEST"
!unzip -q /content/BraTS_New.zip -d /content/


class MultiTaskDataset(Dataset):
    def_init_(self, image_dir, mask_dir, label_csv):
        self.image_dir = image_dir
        self.mask_dir = mask_dir
        self.df = pd.read_csv(label_csv)

    def_len_(self):
        return len(self.df)

    def load_slice(self, patient, slice_id):
        path = os.path.join(self.image_dir, f"{patient}_{slice_id:03d}.npy")
        if os.path.exists(path):
            return np.load(path)
        else:
            return None
```

```
def__getitem__(self, idx):
    row = self.df.iloc[idx]
    patient = row["patient_id"]
    slice_id = row["slice_id"]
    label = row["tumor_present"]

    # Load 3 slices (prev, current, next)
    slices = []
    for sid in [slice_id - 1, slice_id, slice_id + 1]:
        s = self.load_slice(patient, sid)
        if s is None:
            s = self.load_slice(patient, slice_id)
        slices.append(s)

    image = np.concatenate(slices, axis=2) # (H, W, 12)

    mask_path = os.path.join(self.mask_dir, f"{patient}_{slice_id:03d}_mask.png")
    mask = cv2.imread(mask_path, 0) / 255.0

    # Resize (CRITICAL)
    image = cv2.resize(image, (128, 128))
    mask = cv2.resize(mask, (128, 128))

    #  Normalize
    image = image / (np.max(image) + 1e-8)

    #  Augmentation
    if np.random.rand() > 0.5:
        image = np.flip(image, axis=1).copy()
        mask = np.flip(mask, axis=1).copy()

    image = torch.tensor(image, dtype=torch.float32).permute(2,0,1) # (12,H,W)
    mask = torch.tensor(mask, dtype=torch.float32).unsqueeze(0)
    label = torch.tensor(label, dtype=torch.float32)

    return image, mask, label
TRAIN_IMG = "/content/BraTS_New/train/images"
TRAIN_MASK = "/content/BraTS_New/train/masks"
TRAIN_CSV = "/content/BraTS_New/train/labels.csv"
```

```
VAL_IMG = "/content/BraTS_New/val/images"
VAL_MASK = "/content/BraTS_New/val/masks"
VAL_CSV = "/content/BraTS_New/val/labels.csv"

train_dataset = MultiTaskDataset(TRAIN_IMG, TRAIN_MASK, TRAIN_CSV)
val_dataset = MultiTaskDataset(VAL_IMG, VAL_MASK, VAL_CSV)

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False)

vgg = models.vgg16(weights=models.VGG16_Weights.DEFAULT)

old_w = vgg.features[0].weight
vgg.features[0] = nn.Conv2d(12, 64, 3, padding=1)

with torch.no_grad():
    vgg.features[0].weight[:, :3] = old_w
    vgg.features[0].weight[:, 3:] = old_w[:, :1].repeat(1,9,1,1)

features = vgg.features.to(device)

yolo_conv = nn.Conv2d(512, 256, 3, padding=1).to(device)
yolo_bn = nn.BatchNorm2d(256).to(device)
yolo_fc = nn.Linear(256, 1).to(device)

def yolo_forward(x):
    x = F.silu(yolo_bn(yolo_conv(x)))
    x = F.adaptive_avg_pool2d(x, 1).view(x.size(0), -1)
    return torch.sigmoid(yolo_fc(x)).squeeze()



hdsa_d1 = nn.Conv2d(512, 512, 3, padding=1, dilation=1).to(device)
hdsa_d2 = nn.Conv2d(512, 512, 3, padding=2, dilation=2).to(device)
hdsa_d3 = nn.Conv2d(512, 512, 3, padding=3, dilation=3).to(device)

hdsa_attn = nn.Conv2d(512, 1, 1).to(device)

def hdsa_forward(x):
    d1 = hdsa_d1(x)
    d2 = hdsa_d2(x)
    d3 = hdsa_d3(x)

    fused = d1 + d2 + d3
    attn = torch.sigmoid(hdsa_attn(fused))
```

```
    return fused * attn

up1 = nn.ConvTranspose2d(512, 256, 2, 2).to(device)
conv1 = nn.Conv2d(256, 256, 3, padding=1).to(device)

up2 = nn.ConvTranspose2d(256, 128, 2, 2).to(device)
conv2 = nn.Conv2d(128, 128, 3, padding=1).to(device)

up3 = nn.ConvTranspose2d(128, 64, 2, 2).to(device)
conv3 = nn.Conv2d(64, 64, 3, padding=1).to(device)

seg_out = nn.Conv2d(64, 1, 1).to(device)

# Skip connection channel alignment
skip4 = nn.Conv2d(512, 256, 1).to(device)
skip3 = nn.Conv2d(256, 128, 1).to(device)
skip2 = nn.Conv2d(128, 64, 1).to(device)

def forward_pass(x):

    f1 = features[:5](x)
    f2 = features[5:10](f1)
    f3 = features[10:17](f2)
    f4 = features[17:24](f3)
    f5 = features[24:](f4)

    cls = yolo_forward(f5)

    h = hdsa_forward(f5)

    s = F.relu(conv1(up1(h)))
    s = s + skip4(f4)

    s = F.relu(conv2(up2(s)))
    s = s + skip3(f3)

    s = F.relu(conv3(up3(s)))
    s = s + skip2(f2)

    seg = torch.sigmoid(seg_out(s))
    seg = F.interpolate(seg, size=x.shape[2:], mode='bilinear', align_corners=False)

    return cls, seg
def dice_loss(pred, target):
```

```
    # FIX: resize prediction to match target
    if pred.shape != target.shape:
        pred = F.interpolate(pred, size=target.shape[2:], mode='bilinear', align_corners=False)

    pred = pred.view(-1)
    target = target.view(-1)

    inter = (pred * target).sum()
    return 1 - (2 * inter + 1) / (pred.sum() + target.sum() + 1)
def compute_metrics(cls_p, cls_t, seg_p, seg_t):

    # Ensure same size
    if seg_p.shape != seg_t.shape:
        seg_p = F.interpolate(seg_p, size=seg_t.shape[2:], mode='bilinear', align_corners=False)

    acc = ((cls_p > 0.5) == cls_t).float().mean()

    seg_p = (seg_p > 0.5).float()
    inter = (seg_p * seg_t).sum()
    union = seg_p.sum() + seg_t.sum() - inter

    dice = (2 * inter) / (seg_p.sum() + seg_t.sum() + 1e-6)
    iou = inter / (union + 1e-6)

    return acc.item(), dice.item(), iou.item()

def focal_loss(pred, target, alpha=0.8, gamma=2):
    bce = F.binary_cross_entropy(pred, target, reduction='none')
    pt = torch.exp(-bce)
    return (alpha * (1 - pt) ** gamma * bce).mean()

def RAL_loss(cls_p, cls_t, seg_p, seg_t, x):

    bce = nn.BCELoss()

    # Classification
    L_cls = bce(cls_p, cls_t)

    # Strong segmentation loss
    weighted_bce = nn.BCELoss(weight=(seg_t * 6 + 1))
    L_seg = 0.4 * dice_loss(seg_p, seg_t) + \
            0.3 * weighted_bce(seg_p, seg_t) + \
            0.3 * focal_loss(seg_p, seg_t)

    density = torch.mean(seg_p)
```

```
    L_density = torch.abs(density - 0.5)

    return 0.2 * L_cls + 0.8 * L_seg + 0.1 * L_density

params = list(features.parameters()) + \
        list(yolo_conv.parameters()) + list(yolo_bn.parameters()) + list(yolo_fc.parameters()) + \
        list(hdsa_d1.parameters()) + list(hdsa_d2.parameters()) + list(hdsa_d3.parameters()) + \
        list(hdsa_attn.parameters()) + \
        list(up1.parameters()) + list(up2.parameters()) + list(up3.parameters()) + \
        list(seg_out.parameters())
params += list(skip4.parameters()) + list(skip3.parameters()) + list(skip2.parameters())

optimizer = torch.optim.Adam(params, lr=LR)

def train_epoch(loader):
    features.train()
    total = 0

    for x, m, y in loader:
        x, m, y = x.to(device), m.to(device), y.to(device)

        optimizer.zero_grad()

        cls, seg = forward_pass(x)

        loss = RAL_loss(cls, y, seg, m, x)
        loss.backward()

        optimizer.step()

        total += loss.item()

    return total / len(loader)

def validate(loader):
    features.eval()

    accs, dices, ious = [], [], []

    with torch.no_grad():
        for x, m, y in loader:
            x, m, y = x.to(device), m.to(device), y.to(device)

            cls, seg = forward_pass(x)
```

```
            acc, dice, iou = compute_metrics(cls, y, seg, m)

            accs.append(acc)
            dices.append(dice)
            ious.append(iou)

    return np.mean(accs), np.mean(dices), np.mean(ious)

activations = None
gradients = None

def fwd_hook(m, i, o):
    global activations
    activations = o

def bwd_hook(m, gi, go):
    global gradients
    gradients = go[0]

features[-1].register_forward_hook(fwd_hook)
features[-1].register_full_backward_hook(bwd_hook)

def gradcam_plus_plus(x):
    cls, _ = forward_pass(x)
    cls.mean().backward()

    w = gradients.mean(dim=(2,3), keepdim=True)
    cam = (w * activations).sum(dim=1)

    cam = F.relu(cam)
    cam = F.interpolate(cam.unsqueeze(1), size=x.shape[2:], mode='bilinear')

    return cam.squeeze().detach().cpu().numpy()

train_losses = []
val_dices = []

for epoch in range(EPOCHS):
    loss = train_epoch(train_loader)
    acc, dice, iou = validate(val_loader)

    train_losses.append(loss)
    val_dices.append(dice)

    print(f"Epoch {epoch+1}")
```

```
    print(f"Loss: {loss:.4f}")
    print(f"Acc: {acc:.4f} | Dice: {dice:.4f} | IoU: {iou:.4f}")

sample = next(iter(val_loader))[0][0].unsqueeze(0).to(device)

heatmap = gradcam_plus_plus(sample)

import matplotlib.pyplot as plt
plt.imshow(heatmap, cmap='jet')
plt.title("Grad-CAM++")
plt.colorbar()
plt.show()

import matplotlib.pyplot as plt
import cv2

# Get sample
images, masks, labels = next(iter(val_loader))

img = images[0].cpu().numpy()      # (4, H, W)
mask = masks[0].cpu().numpy()      # (1, H, W)

# Use one modality for visualization (FLAIR is best usually)
base_img = img[3] # flair channel

# Normalize image for display
base_img = (base_img - base_img.min()) / (base_img.max() - base_img.min() + 1e-8)

# Prepare input
input_tensor = images[0].unsqueeze(0).to(device)

# Generate Grad-CAM
heatmap = gradcam_plus_plus(input_tensor)

# Normalize heatmap
heatmap = (heatmap - heatmap.min()) / (heatmap.max() - heatmap.min() + 1e-8)

# Resize heatmap to image size
heatmap_resized = cv2.resize(heatmap, (base_img.shape[1], base_img.shape[0]))

# Convert to color map
heatmap_color = cv2.applyColorMap(np.uint8(255 * heatmap_resized), cv2.COLORMAP_JET)
heatmap_color = cv2.cvtColor(heatmap_color, cv2.COLOR_BGR2RGB) / 255.0

# Convert base image to RGB
```

```
base_rgb = np.stack([base_img]*3, axis=-1)

# Overlay
overlay = 0.6 * base_rgb + 0.4 * heatmap_color

# Plot
plt.figure(figsize=(15,5))

plt.subplot(1,4,1)
plt.imshow(base_img, cmap='gray')
plt.title("Original MRI (FLAIR)")
plt.axis('off')

plt.subplot(1,4,2)
plt.imshow(mask.squeeze(), cmap='gray')
plt.title("Ground Truth Mask")
plt.axis('off')

plt.subplot(1,4,3)
plt.imshow(heatmap_resized, cmap='jet')
plt.title("Grad-CAM++ Heatmap")
plt.axis('off')

plt.subplot(1,4,4)
plt.imshow(overlay)
plt.title("Overlay (MRI + Grad-CAM++)")
plt.axis('off')

plt.show()

all_preds = []
all_labels = []

features.eval()

with torch.no_grad():
    for x, m, y in val_loader:
        x = x.to(device)

        cls, _ = forward_pass(x)

        all_preds.extend(cls.cpu().numpy())
        all_labels.extend(y.numpy())

all_preds = np.array(all_preds)
```

```
all_labels = np.array(all_labels)

threshold = 0.5
pred_binary = (all_preds > threshold).astype(int)


from sklearn.metrics import confusion_matrix

tn, fp, fn, tp = confusion_matrix(all_labels, pred_binary).ravel()

accuracy = (tp + tn) / (tp + tn + fp + fn)
precision = tp / (tp + fp + 1e-8)
recall = tp / (tp + fn + 1e-8) # Sensitivity
specificity = tn / (tn + fp + 1e-8)
f1 = 2 * precision * recall / (precision + recall + 1e-8)

print(f"Accuracy     : {accuracy:.4f}")
print(f"Precision    : {precision:.4f}")
print(f"Recall (Sens): {recall:.4f}")
print(f"Specificity : {specificity:.4f}")
print(f"F1 Score     : {f1:.4f}")

import matplotlib.pyplot as plt
import seaborn as sns

cm = confusion_matrix(all_labels, pred_binary)

plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=["No Tumor", "Tumor"],
            yticklabels=["No Tumor", "Tumor"])

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()

from sklearn.metrics import roc_curve, auc

fpr, tpr, _ = roc_curve(all_labels, all_preds)
roc_auc = auc(fpr, tpr)

plt.figure()
plt.plot(fpr, tpr, label=f"AUC = {roc_auc:.4f}")
plt.plot([0,1], [0,1], linestyle='--')
```

```
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()
plt.show()

metrics = [accuracy, precision, recall, specificity, f1]
names = ["Accuracy", "Precision", "Recall", "Specificity", "F1"]

plt.figure(figsize=(8,4))
plt.bar(names, metrics)
plt.title("Evaluation Metrics")
plt.show()

import matplotlib.pyplot as plt

plt.figure(figsize=(8,5))

plt.plot(train_losses, label="Training Loss", marker='o')

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training Loss Curve")
plt.legend()
plt.grid()

plt.show()

plt.figure(figsize=(8,5))

plt.plot(val_dices, label="Validation Dice Score", marker='o')

plt.xlabel("Epoch")
plt.ylabel("Dice Score")
plt.title("Dice Score vs Epoch")
plt.legend()
plt.grid()

plt.show()
```

5. SYSTEM TESTING

6.1 Overview of Testing

Software Testing is evaluation of the software against requirements gathered from users and system specifications. Testing is conducted at the phase level in software development life cycle or at module level in program code. Software testing comprises of Validation and Verification.

6.2 Types of Tests

6.2.2 Unit Testing

UNIT TESTING is a level of software testing where individual units/ components ofa softwarearetested.Thepurposeistovalidatethateachunitofthesoftwareperformsasdesigned.A unit is the smallest testable part of any software. It usually has one or a few inputs and usually asingle output.

6.2.3 Integration Testing

INTEGRATION TESTING is a level of software testing where individual units are combined and tested as a group. The purpose of this level of testing is to expose faults in the interaction between integrated units. Test drivers and test stubs are used to assist in Integration Testing.

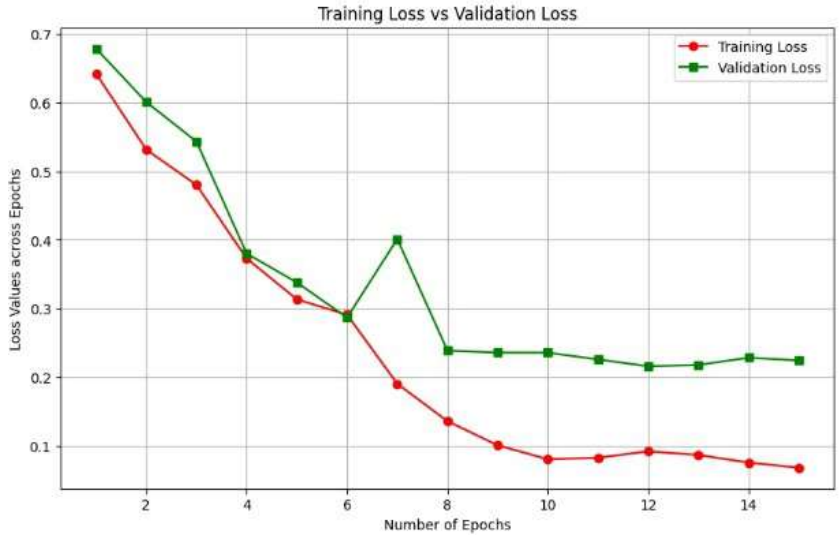
6. RESULTS

Table 1. Accuracy and loss for the model: CNN.

BASELINE-CNN				
Epochs	Performance Accuracy		Performance Loss	
	Training Accuracy	Validation Accuracy	Training Loss	Validation Loss
1	0.6587	0.56	0.6314	0.7161
2	0.6636	0.56	0.6118	0.7050
3	0.7974	0.60	0.4267	0.5973
4	0.8484	0.68	0.3571	0.5374
5	0.9185	0.84	0.2386	0.4231
6	0.9643	0.84	0.1774	0.4167
7	0.9719	0.84	0.1160	0.3754
8	0.9857	0.84	0.0698	0.4159
9	0.9785	0.84	0.0807	0.4006
10	0.9912	0.80	0.0388	0.4637

Table 2. Accuracy and loss for the model: VGG-16.

VGG-16				
Epochs	Performance Accuracy		Performance Loss	
	Training Accuracy	Validation Accuracy	Training Loss	Validation Loss
1	0.637475	0.55	0.642475	0.678875
2	0.714275	0.67	0.531650	0.601000
3	0.746975	0.74	0.480125	0.543325
4	0.849375	0.84	0.372900	0.380075
5	0.869150	0.85	0.313300	0.337925
6	0.888275	0.88	0.290825	0.286575
7	0.924000	0.83	0.190250	0.401275
8	0.961325	0.92	0.135725	0.238600
9	0.954326	0.91	0.100498	0.235640
10	0.973325	0.92	0.080175	0.235550
11	0.973050	0.91	0.082400	0.225700
12	0.971175	0.92	0.091675	0.215725
13	0.971954	0.93	0.086382	0.217680
14	0.970950	0.94	0.075350	0.228250
15	0.982625	0.94	0.067850	0.224250



31 Fig 9. VGG-16 Training Loss VS Validation Loss

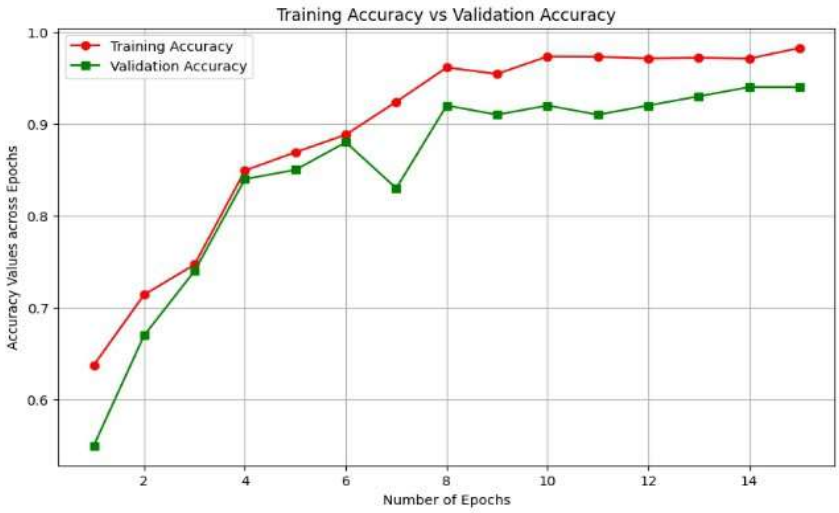


Fig 10.VGG-16 Training Accuracy VS Validation Accuracy

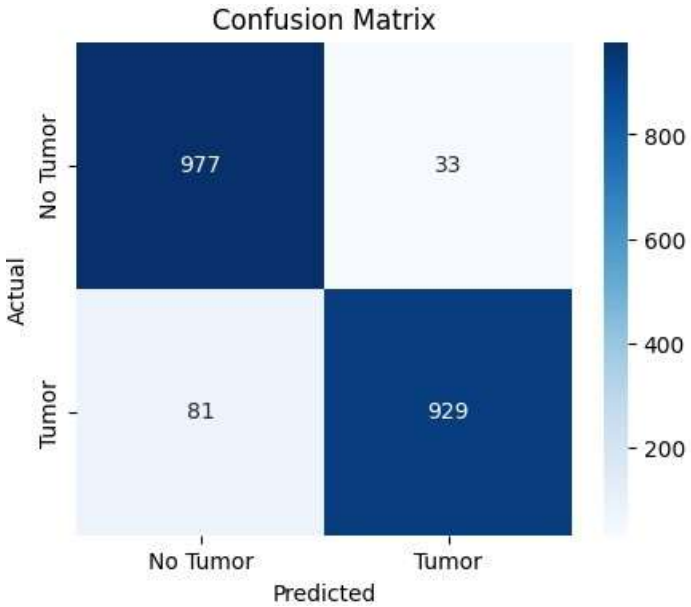


Fig 11. Confusion Matrix for VGG-16

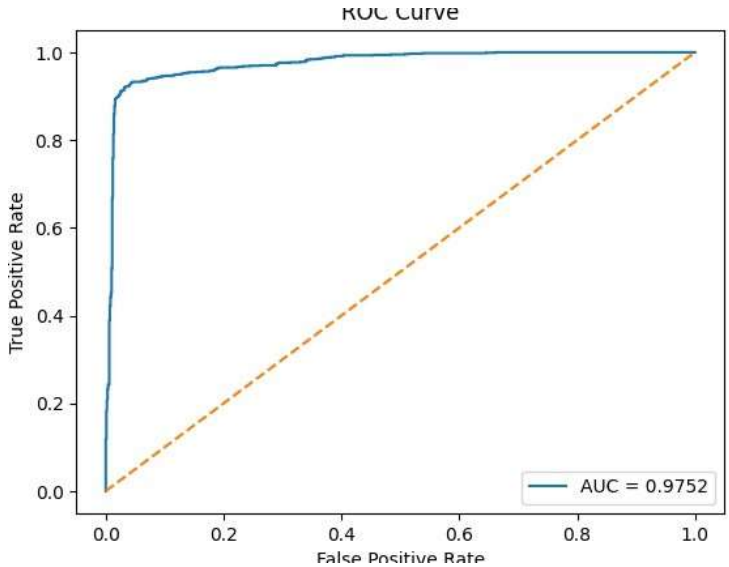


Fig 12. ROC Graph for VGG-16

7. CONCLUSION

Ultimately, this work introduces a deep learning approach capable of processing MRI scans in real time, regardless of image resolution, to detect and assess glioma tumors. Built around an adapted VGG-16 structure, it integrates YOLO-inspired detection with HDSA-driven segmentation into one cohesive pipeline. Because these elements function in tandem, localization of tumor areas occurs alongside precise boundary delineation, without sacrificing operational speed. Notably, the VGG-16 component delivers reliable output, outperforming standard CNNs through improved accuracy and reduced variability across tests.

Despite handling several jobs at once, the framework keeps performance steady. Detection alongside segmentation happens in one go - no splitting needed. Better precision comes through, along with reduced error rates. Generalization improves too, standing above older methods. Reliability grows, yes - but so does ease in daily medical settings. What looks complex works simply where it counts.

A key aspect lies in the Resolution-Agnostic Loss (RAL), letting performance stay stable despite differences in MRI scan clarity or detail. Since medical imaging environments often deliver uneven results, flexibility here becomes highly useful. Instead of relying solely on raw inputs, it benefits from steps like standardization and filtering - methods that clean up data before analysis begins. These adjustments open space for better pattern recognition during later stages.

Although built for one purpose, the system adjusts well when faced with intricate features hidden in MRI data. Because of this flexibility, application spans multiple types of patient records and hospital settings. Even when scan quality shifts or machines differ, results stay consistent - proof of reliability beyond controlled tests.

Deep learning finds practical use in medical imaging through this study. Accuracy meets efficiency here, thanks to a design built for growth and everyday hurdles. Progress toward dependable AI in clinics becomes clearer when tools support physicians with timely insights. Decisions gain speed without losing depth, simply because systems now align better with clinical needs.

8. REFERENCES

[1] V. Rajinikanth, S. Kadry, R. Damaševičius, R. A. Sujitha, G. Balaji, and M. A. Mohammed, "Glioma/glioblastoma detection in brain MRI using pre-trained deep-learning scheme," in Proc. 3rd Int. Conf. Intelligent Computing Instrumentation and Control Technologies (ICICICT), Kannur, India, 2022, pp. 987–990, doi: 10.1109/ICICICT54557.2022.9917904.

[2] M. K. Abd-Ellah, A. I. Awad, H. F. A. Hamed, and A. A. M. Khalaf, "Parallel deep CNN structure for glioma detection and classification via brain MRI images," in Proc. 31st Int. Conf. Microelectronics (ICM), Cairo, Egypt, 2019, pp. 304–307, doi: 10.1109/ICM48031.2019.9021872.

[3] H. A. Hafeez *et al.*, "A CNN-model to classify low-grade and high-grade glioma from MRI images," *IEEE Access*, vol. 11, pp. 46283–46296, 2023, doi: 10.1109/ACCESS.2023.3273487.

[4] P. C. Tripathi and S. Bag, "An attention-guided CNN framework for segmentation and grading of glioma using 3D MRI scans," IEEE/ACM Transactions on Computational Biology and Bioinformatics, vol. 20, no. 3, pp. 1890–1904, May–Jun. 2023, doi: 10.1109/TCBB.2022.3220902.

[5] H. N. T. K. Kaldera, S. R. Gunasekara, and M. B. Dissanayake, "MRI-based glioma segmentation using deep learning algorithms," in Proc. Int. Research Conf. Smart Computing and Systems Engineering (SCSE), Colombo, Sri Lanka, 2019, pp. 51–56, doi: 10.23919/SCSE.2019.8842668.

[6] R. Vinoth and C. Venkatesh, "Segmentation and detection of tumor in MRI images using CNN and SVM classification," in Proc. Conf. Emerging Devices and Smart Systems (ICEDSS), Tiruchengode, India, 2018, pp. 21–25, doi: 10.1109/ICEDSS.2018.8544306.

[7] M. Rizwan, A. Shabbir, A. R. Javed, M. Shabbir, T. Baker, and D. Al-Jumeily Obe, "Brain tumor and glioma grade classification using Gaussian convolutional neural network," IEEE Access, vol. 10, pp. 29731–29740, 2022, doi: 10.1109/ACCESS.2022.3153108.

[8] A. B. Abdusalomov, M. Mukhiddinov, and T. K. Whangbo, "Brain tumor detection based on

deep learning approaches and magnetic resonance imaging," Cancers, vol. 15, no. 16, p. 4172, Aug. 2023.

[9] A. A. Asiri, M. A. Alzahrani, S. A. Alshahrani, and S. A. Alqahtani, "Brain tumor detection and classification using fine-tuned CNN with ResNet50 and U-Net model," Life, vol. 13, no. 7, p. 1449, Jun. 2023.

[10] S. Saeedi, S. Rezayi, H. Keshavarz, and S. R. N. Kalhori, "MRI-based brain tumor detection using convolutional deep learning methods and selected machine learning techniques," BMC Medical Informatics and Decision Making, vol. 23, no. 114, 2023.

[11] A. Chattopadhyay and M. Maitra, "MRI-based brain tumor image detection using CNN-based deep learning method," Neuroscience Informatics, vol. 2, p. 100060, 2022.

[12] H. ZainEldin, S. A. Gamel, E.-S. M. El-Kenawy, A. H. Alharbi, D. S. Khafaga, A. Ibrahim, and F. M. Talaat, "Brain tumor detection and classification using deep learning and sine-cosine fitness grey wolf optimization," Bioengineering, vol. 10, no. 1, p. 18, Dec. 2022.

[13] M. N. Islam, M. M. Rahman, and M. A. Hossain, "An improved deep learning-based hybrid model with ensemble techniques for brain tumor detection from MRI images," Informatics in Medicine Unlocked, vol. 47, p. 101483, 2024.

[14] N. Rasool, M. A. Khan, A. Rehman, and S. Kadry, "CNN-TumorNet: Leveraging explainability in deep learning for precise brain tumor diagnosis on MRI images," Frontiers in Oncology, vol. 15, p. 1554559, 2025.

[15] T. L. Prasanthi and N. Neelima, "Improvement of brain tumor categorization using deep learning: A comprehensive investigation and comparative analysis," Procedia Computer Science, vol. 233, pp. 703–712, 2024.

[16] Y. Bhanothu, A. Kamalakannan, and G. Rajamanickam, "Detection and classification of brain tumor in MRI images using deep convolutional network," in Proc. 6th Int. Conf. Advanced Computing and Communication Systems (ICACCS), Coimbatore, India, 2020, pp. 248–252, doi: 10.1109/ICACCS48705.2020.9074375.